

Programmazione a oggetti

Classi e oggetti

Come rappresentare entità del programma con classi, campi e oggetti.

L'idea della programmazione a oggetti

Java è un linguaggio molto legato alla programmazione a oggetti.

Un **oggetto** rappresenta qualcosa con dati e comportamenti: una persona, un prodotto, un conto, un libro.

Una **classe** è il modello da cui crei gli oggetti.

Pensala così:

- la classe è il modulo vuoto
- l'oggetto è un modulo compilato con dati reali

Creare una classe

```
public class Persona {  
    String nome;  
    int eta;  
}
```

Questa classe descrive una persona con due campi:

- `nome`
- `eta`

I campi sono variabili che appartengono a un oggetto.

Creare un oggetto con new

```
public class Esempio {  
    public static void main(String[] args) {  
        Persona persona = new Persona();  
  
        persona.nome = "Luca";  
        persona.eta = 20;  
  
        System.out.println(persona.nome);  
        System.out.println(persona.eta);  
    }  
}
```

Output:

```
Luca  
20
```

`new Persona()` crea un nuovo oggetto partendo dalla classe `Persona` .

Classe e oggetto non sono la stessa cosa

La classe è il progetto.

L'oggetto è una cosa concreta creata da quel progetto.

```
Persona persona1 = new Persona();  
Persona persona2 = new Persona();  
  
persona1.nome = "Luca";  
persona2.nome = "Sara";
```

`persona1` e `persona2` sono due oggetti diversi. Hanno la stessa struttura, ma possono contenere dati diversi.

Aggiungere un metodo

Una classe può contenere anche metodi.

```
public class Persona {  
    String nome;  
    int eta;  
  
    void saluta() {  
        System.out.println("Ciao, sono " + nome);  
    }  
}
```

Ora puoi chiamare il metodo sull'oggetto:

```
Persona persona = new Persona();  
persona.nome = "Luca";  
  
persona.saluta();
```

Output:

```
Ciao, sono Luca
```

Il metodo usa il campo `nome` dell'oggetto su cui viene chiamato.

Un esempio con prodotti

```
public class Prodotto {  
    String nome;  
    double prezzo;  
  
    void mostra() {  
        System.out.println(nome + ": " + prezzo + " euro");  
    }  
}
```

Uso:

```
public class Negozio {  
    public static void main(String[] args) {  
        Prodotto prodotto = new Prodotto();  
        prodotto.nome = "Quaderno";  
        prodotto.prezzo = 2.50;  
  
        prodotto.mostra();  
    }  
}
```

Output:

```
Quaderno: 2.5 euro
```

Perche usare classi

Le classi aiutano a tenere insieme dati collegati.

Senza classe potresti avere variabili separate:

```
String nomeProdotto = "Quaderno";  
double prezzoProdotto = 2.50;
```

Con una classe, il concetto diventa più chiaro:

```
Prodotto prodotto = new Prodotto();
```

Man mano che i programmi crescono, questa organizzazione diventa fondamentale.

Costruttori

Come inizializzare un oggetto quando viene creato.

Il problema da risolvere

Nella pagina precedente creavamo un oggetto e poi riempivamo i campi:

```
Persona persona = new Persona();  
persona.nome = "Luca";  
persona.eta = 20;
```

Funziona, ma è facile dimenticare un campo.

Un **costruttore** serve a inizializzare l'oggetto nel momento in cui viene creato.

Creare un costruttore

```
public class Persona {  
    String nome;  
    int eta;  
  
    public Persona(String nome, int eta) {  
        this.nome = nome;  
        this.eta = eta;  
    }  
}
```

Il costruttore ha lo stesso nome della classe.

Non ha tipo di ritorno: non scrivi ne `void` ne `int` ne altro.

Usare il costruttore

```
public class Esempio {  
    public static void main(String[] args) {  
        Persona persona = new Persona("Luca", 20);  
  
        System.out.println(persona.nome);  
        System.out.println(persona.eta);  
    }  
}
```

Output:

```
Luca  
20
```

I valori `"Luca"` e `20` vengono passati al costruttore.

A cosa serve this

Nel costruttore abbiamo scritto:

```
this.nome = nome;
```

Qui ci sono due `nome` :

- `this.nome` e il campo dell'oggetto
- `nome` e il parametro del costruttore

`this` significa "questo oggetto".

La riga si legge così: "metti nel campo `nome` di questo oggetto il valore ricevuto nel parametro `nome`".

Costruttore predefinito

Se non scrivi nessun costruttore, Java ne crea uno vuoto per te.

```
public class Persona {  
    String nome;  
    int eta;  
}
```

Puoi fare:

```
Persona persona = new Persona();
```

Ma appena scrivi un costruttore con parametri, Java non crea più automaticamente quello vuoto.

Piu costruttori

Puoi avere più costruttori con parametri diversi.

```

public class Persona {
    String nome;
    int eta;

    public Persona() {
        nome = "Sconosciuto";
        eta = 0;
    }

    public Persona(String nome, int eta) {
        this.nome = nome;
        this.eta = eta;
    }
}

```

Uso:

```

Persona a = new Persona();
Persona b = new Persona("Sara", 25);

```

Questo è un caso di overloading applicato ai costruttori.

Esempio completo

```

public class Prodotto {
    String nome;
    double prezzo;

    public Prodotto(String nome, double prezzo) {
        this.nome = nome;
        this.prezzo = prezzo;
    }

    void mostra() {
        System.out.println(nome + ": " + prezzo + " euro");
    }
}

```



```
public class Negozio {  
    public static void main(String[] args) {  
        Prodotto quaderno = new Prodotto("Quaderno", 2.50);  
        Prodotto penna = new Prodotto("Penna", 1.20);  
  
        quaderno.mostra();  
        penna.mostra();  
    }  
}
```

I costruttori rendono più difficile creare oggetti incompleti.

Incapsulamento

Come proteggere i dati di un oggetto con private, getter e setter.

Perché non lasciare tutto pubblico

Finora abbiamo modificato i campi direttamente:

```
persona.eta = -5;
```

Java lo permette se il campo è accessibile, ma questo valore non ha senso.

L'**incapsulamento** serve a proteggere i dati di un oggetto e controllare come vengono letti o modificati.

private

Con `private`, un campo può essere usato solo dentro la classe.

```
public class Persona {  
    private String nome;  
    private int eta;  
}
```

Da fuori non puoi più fare:

```
persona.eta = 20; // errore se eta e private
```

Serve un modo controllato per leggere e modificare il valore.

Getter

Un getter restituisce il valore di un campo.

```
public String getNome() {  
    return nome;  
}
```

Esempio:

```
public class Persona {  
    private String nome;  
  
    public String getNome() {  
        return nome;  
    }  
}
```

`getNome` permette di leggere `nome` senza rendere il campo pubblico.

Setter

Un setter modifica il valore di un campo.

```
public void setName(String nome) {  
    this.nome = nome;  
}
```

Il vantaggio è che puoi controllare il valore prima di salvarlo.

```
public void setEta(int eta) {  
    if (eta >= 0) {  
        this.eta = eta;  
    }  
}
```

Se qualcuno prova a impostare `-5`, il valore non viene accettato.

Classe completa

```
public class Persona {  
    private String nome;  
    private int eta;  
  
    public Persona(String nome, int eta) {  
        this.nome = nome;  
        setEta(eta);  
    }  
  
    public String getNome() {  
        return nome;  
    }  
  
    public int getEta() {  
        return eta;  
    }  
  
    public void setEta(int eta) {  
        if (eta >= 0) {  
            this.eta = eta;  
        }  
    }  
}
```

Uso:

```
public class Esempio {  
    public static void main(String[] args) {  
        Persona persona = new Persona("Luca", 20);  
  
        System.out.println(persona.getNome());  
        System.out.println(persona.getEta());  
  
        persona.setEta(21);  
        System.out.println(persona.getEta());  
    }  
}
```

Output:

```
Luca
20
21
```

public e private

`public` significa accessibile dall'esterno.

`private` significa accessibile solo dentro la classe.

Una regola molto comune in Java e:

- campi `private`
- metodi pubblici solo quando servono

Perche e utile

L'incapsulamento aiuta a:

- evitare valori non validi
- cambiare l'interno della classe senza rompere tutto il programma
- rendere chiaro quali azioni sono permesse

All'inizio sembra piu lungo. Nei programmi reali rende il codice piu sicuro e ordinato.

Ereditarieta

Come creare classi specializzate partendo da classi piu generali.

Cosa significa ereditarieta

L'ereditarieta permette di creare una classe partendo da un'altra.

La classe piu generale contiene dati e metodi comuni. La classe piu specifica li riusa e aggiunge cio che le serve.

Esempio:

- `Persona` e generale

- `Studente` è una persona con una matricola

extends

In Java si usa `extends` .

```
public class Persona {  
    String nome;  
  
    void saluta() {  
        System.out.println("Ciao, sono " + nome);  
    }  
}
```

```
public class Studente extends Persona {  
    String matricola;  
}
```

`Studente` eredita il campo `nome` e il metodo `saluta` .

Usare la classe derivata

```
public class Esempio {  
    public static void main(String[] args) {  
        Studente studente = new Studente();  
  
        studente.nome = "Luca";  
        studente.matricola = "A123";  
  
        studente.saluta();  
        System.out.println(studente.matricola);  
    }  
}
```

Output:

Ciao, sono Luca
A123

Classe base e classe derivata

La classe da cui parti viene spesso chiamata:

- classe base
- superclasse
- classe padre

La classe che eredita viene chiamata:

- classe derivata
- sottoclasse
- classe figlia

Sono parole diverse per descrivere la stessa relazione.

Sovrascrivere un metodo

Una sottoclasse può ridefinire un metodo ereditato.

```
public class Studente extends Persona {  
    String matricola;  
  
    @Override  
    void saluta() {  
        System.out.println("Ciao, sono lo studente " + nome);  
    }  
}
```

`@Override` dice a Java e a chi legge: "sto sovrascrivendo un metodo della classe base".

Costruttori e super

Se la classe base ha un costruttore con parametri, la sottoclasse deve chiamarlo con `super`.

```
public class Persona {  
    private String nome;  
  
    public Persona(String nome) {  
        this.nome = nome;  
    }  
  
    public String getNome() {  
        return nome;  
    }  
}
```

```
public class Studente extends Persona {  
    private String matricola;  
  
    public Studente(String nome, String matricola) {  
        super(nome);  
        this.matricola = matricola;  
    }  
}
```

`super(nome)` chiama il costruttore di `Persona` .

Quando usarla con attenzione

L'ereditarietà è utile quando una classe è davvero un tipo più specifico di un'altra.

`Studente` è una `Persona` : ha senso.

Ma non usarla solo per "prendere codice". A volte è meglio mettere un oggetto dentro un altro invece di ereditarlo.

Regola pratica:

se puoi dire "X è un tipo di Y", l'ereditarietà potrebbe avere senso.

Se devi dire "X usa Y" o "X contiene Y", probabilmente serve un'altra struttura.

Interfacce

Come definire comportamenti comuni che classi diverse possono implementare.

Cosa è un'interfaccia

Un'interfaccia descrive un comportamento che una classe promette di avere.

Non dice necessariamente come farlo. Dice quali metodi devono esistere.

Pensala come un contratto:

"se una classe implementa questa interfaccia, allora deve offrire questi metodi".

Creare un'interfaccia

```
public interface Stampabile {  
    void stampa();  
}
```

Questa interfaccia richiede un metodo `stampa` .

Una classe che la implementa deve scrivere quel metodo.

implements

```
public class Documento implements Stampabile {  
    private String titolo;  
  
    public Documento(String titolo) {  
        this.titolo = titolo;  
    }  
  
    @Override  
    public void stampa() {  
        System.out.println("Documento: " + titolo);  
    }  
}
```

`implements Stampabile` significa: `Documento` rispetta il contratto `Stampabile`.

Classi diverse, stesso comportamento

```
public class Foto implements Stampabile {  
    private String nomeFile;  
  
    public Foto(String nomeFile) {  
        this.nomeFile = nomeFile;  
    }  
  
    @Override  
    public void stampa() {  
        System.out.println("Foto: " + nomeFile);  
    }  
}
```

`Documento` e `Foto` sono classi diverse, ma entrambe sanno stampare.

Usare il tipo interfaccia

Puoi usare l'interfaccia come tipo.

```
public class Esempio {
    public static void main(String[] args) {
        Stampabile a = new Documento("Contratto");
        Stampabile b = new Foto("vacanza.jpg");

        a.stampa();
        b.stampa();
    }
}
```

Output:

```
Documento: Contratto
Foto: vacanza.jpg
```

Il programma non ha bisogno di sapere tutti i dettagli interni. Gli basta sapere che entrambi sono `Stampabile`.

Perche sono utili

Le interfacce aiutano a scrivere codice flessibile.

Puoi dire: "mi serve qualcosa che sappia fare questa azione", senza fissarti su una classe specifica.

Esempio:

```
public static void stampaOggetto(Stampabile oggetto) {
    oggetto.stampa();
}
```

Questo metodo funziona con qualunque classe che implementa `Stampabile`.

Interfacce ed ereditarieta

Una classe Java puo estendere una sola classe:

```
public class Studente extends Persona {  
}
```

Ma puoi implementare più interfacce:

```
public class Documento implements Stampabile, Salvabile {  
}
```

Questo rende le interfacce molto utili per descrivere comportamenti comuni.

Regola pratica

Usa un'interfaccia quando classi diverse devono condividere lo stesso comportamento, anche se non appartengono alla stessa famiglia.

Documento e **Foto** non sono la stessa cosa. Però entrambe possono essere stampabili.